

【 TECH COLUMN 】

지식은 검색되는가, 축적되는가?

Karpathy의 'LLM Wiki'가 드러낸 RAG의 한계, 그리고 그 너머

Dennis Kim (김호광)

*"LLM이 단순히 검색하는 것을 넘어, 읽고, 이해하고, 스스로 정리하도록 만들
어야 한다." — Andrej Karpathy*

2025년 말, OpenAI 창립 멤버이자 전 Tesla AI 디렉터인 안드레이 카파시(Andrej Karpathy)가 던진 한 문장이 AI 엔지니어링 커뮤니티를 흔들었다. 그는 현재 대부분의 AI 애플리케이션이 의존하고 있는 검색 증강 생성(RAG, Retrieval-Augmented Generation)을 '기억상실증 환자에게 매번 메모를 건네주는 방식'에 비유하며, 대안으로 'LLM Wiki'라는 개념을 제안했다. 지식을 매번 검색해 소비하는 것이 아니라, 한 번 읽고 구조화해 '컴파일'한 뒤 축적하자는 것이다.

언뜻 단순해 보이는 이 제안은, 실제로는 지난 2년간 AI 업계가 당연시해온 정보 처리 패러다임 전체를 겨냥한 근본적 문제 제기다. 그리고 그것이 왜 큰 반향을 불러일으켰는지, 또 왜 동시에 냉정한 비판에 직면했는지를 이해하려면, 우리는 먼저 RAG라는 현재의 표준이 어떤 구조적 한계 위에 서 있는지를 들여다볼 필요가 있다.

I. RAG의 '기억상실증' — 왜 다시 '지식'이 문제인가

RAG는 우아한 타협책이었다. LLM의 컨텍스트 창(context window)은 유한하고, 모델을 매번 파인튜닝하기에는 비용과 시간이 너무 크다. 그래서 우리는 외부 지식 베이스에 정보를 저장해두고, 질문이 들어올 때마다 관련 조각을 검색해 프롬프트에 끼워 넣었다. 벡터 임베딩, 하이브리드 서치, 리랭커 — 지난 3년간 AI 인프라 업계는 이 파이프라인을 정교화하는 데 막대한 자원을 쏟아부었다.

그러나 카파시의 지적은 뼈아프다. 이 방식은 '이해'가 아니라 '대조'에 가깝다. 모델은 매 쿼리마다 처음 보는 정보 조각들을 받아 들고 그때그때 조립한다. 어제 답했던 것과 오늘 답하는 것 사이에 학습이 없다. 모순을 발견해도 기록되지 않는다. 동일한 문서를 1000번째 검색해도, 999번째 검색 때와 똑같은 에너지로 다시 읽어야 한다. 인간이라면 결코 이런 식으로 지식과 관계 맺지 않는다.

바로 여기서 '**지식의 컴파일(compilation of knowledge)**'이라는 개념이 등장한다. 소스 코드를 매번 인터프리터로 해석하는 대신 한 번 컴파일해두면, 이후 실행은 빠르고 일관되며 최적화된다. LLM Wiki의 발상 역시 같다. 원시 문서들을 매번 검색해 파편적으로 소비하는 대신, LLM 에이전트가 이들을 읽고, 요약하고, 상호 연결하고, 모순을 표시하여 Markdown 같은 사람이 읽을 수 있는 형태로 정리해둔다. 이후 질문이 들어오면 모델은 이 '정돈된 지식' 위에서 고차원 추론에 집중할 수 있다.

II. 혁신의 실체 — 무엇이 정말 새로운가?

1) 지식의 복리 효과

LLM Wiki가 RAG와 결정적으로 다른 지점은 '시간'을 다루는 방식이다. RAG의 지식 베이스는

선형적으로 커진다. 문서 1000개가 쌓이면 그저 1000개의 독립된 조각이 있을 뿐이다. 반면 LLM Wiki는 새 정보가 들어올 때마다 기존 페이지들과 연결되고, 모순이 생기면 표시되며, 중복은 통합된다. 책 한 권을 읽을 때마다 자동으로 업데이트되는 살아있는 백과사전 — 이것이 카파시가 그린 그림이다.

이 차이는 시간이 지날수록 기하급수적으로 벌어진다. 지식 베이스가 크면 클수록, RAG는 점점 더 노이즈에 시달리지만, LLM Wiki는 이론적으로 점점 더 정교해진다. 이것이 '복리 효과 (compounding effect)'의 의미다.

2) 인간의 인지 부하 재배분

카파시가 이 아이디어의 핵심 가치로 꼽은 것은 흥미롭게도 '성능'이 아니라 '노동 분업'이다. 그는 LLM이 인간을 대신해 '지루한 문서 정리(bookkeeping)' 작업을 해주는 것이 본질이라고 말했다. 정보를 어디서 얻을지 결정하고, 무엇을 탐색할지 정하고, 어떤 질문을 던질지 고민하는 — 이런 고차원적 판단은 인간이 한다. 대신 요약, 상호 참조, 파일 이름 붙이기, 카테고리 정리 같은 저차원 반복 작업은 에이전트가 가져간다.

이 구분은 생성형 AI 시대의 '도구론'에 있어 중요한 전환점을 시사한다. 우리가 LLM에게 기대해야 할 것은 '답을 대신 생각해주는 것'이 아니라, '생각을 위한 인프라를 대신 정돈해주는 것'이라는 관점이다.

3) 투명성과 주도권의 회복

벡터 데이터베이스는 본질적으로 블랙박스다. 3072차원 임베딩 공간에 무엇이 어떻게 저장

되어 있는지, 왜 이 조각이 검색되고 저 조각이 누락됐는지 — 이것을 직관적으로 감사(audit)할 수 있는 엔지니어는 거의 없다. 반면 LLM Wiki는 Markdown 텍스트 파일의 집합일 뿐이다. 사용자는 언제든지 열어보고, 틀린 부분을 고치고, 구조를 재배치할 수 있다. Obsidian 같은 기존 노트 생태계와 바로 결합할 수 있다는 점도 같은 맥락에서 중요하다. 이것은 단순한 편의성 문제가 아니라, 지식에 대한 통제권을 AI 시스템에서 인간에게로 되돌리는 문제다.

Ⅲ. 그림자 — 혁신 뒤에 숨은 네 개의 암초

1) 오류의 누적과 증폭 — 가장 치명적인 아킬레스건

LLM Wiki 구상의 가장 큰 위험은 모델 자체의 불완전성이 구조적으로 증폭된다는 점이다. RAG는 매 질의마다 원본을 참조한다. 할루시네이션이 발생해도 다음 쿼리에서 교정될 기회가 있다. 그러나 LLM Wiki는 다르다. 초기 컴파일 과정에서 모델이 미묘한 뉘앙스를 놓치거나, 특정 관점에 편향되거나, 두 문서 사이의 모순을 잘못 해소하면, 그 오류는 위키에 '사실'로 기록된다.

문제는 여기서 끝나지 않는다. 후속 문서가 들어와 이 오염된 페이지와 상호 참조되기 시작하면, 오류는 한 페이지에 머물지 않고 네트워크 전체로 전이된다. 자가 강화되는 허위(self-reinforcing falsity) — 이것은 위키피디아 같은 인간 편집 시스템에서도 나타나는 문제지만, LLM이 주도하는 시스템에서는 검증자 없이 눈덩이처럼 불어날 수 있다. 최근 Anthropic 및 여러 연구소가 보고한 '모델 간 세대 오염(model collapse)' 현상과 본질적으로 같은 종류의 위험이다.

2) 확장성의 벽

LLM Wiki 접근은 소규모·집중형 지식 베이스에 이상적이다. 그러나 문서가 수천 개를 넘어가

면 경제학이 무너지기 시작한다. 새 문서 하나를 추가할 때마다 에이전트는 전체 위키를 읽고, 관련 페이지를 찾고, 일관성을 검증해야 한다. 실제 벤치마크에서도 노트 **500개 미만에서는 정확도가 양호하지만 1,000개를 넘으면 급격히 저하된다**는 보고가 나오고 있다. 인간 위키피디아 편집자가 700만 개의 문서를 다룰 수 있는 것은 각자가 극히 좁은 영역만 담당하기 때문이다. LLM 에이전트에게는 이런 분업 구조가 아직 없다.

3) 토큰 경제학 — 지구를 따듯하게 하는 기술

컴파일은 공짜가 아니다. 새 자료가 들어올 때마다 위키 전체를 읽고 분석하고 업데이트하는 작업은 막대한 토큰 소비를 수반한다. 위키의 규모가 커질수록 단일 업데이트의 비용은 선형이 아니라 초선형으로 증가한다. 대규모 조직이 이 방식을 도입한다고 가정하면, API 호출 비용만으로 월 수만 달러가 쉽게 나온다. RAG가 단순히 기술적 타협이 아니라 경제적 타협이기도 했다는 점을 다시 상기시키는 대목이다.

4) LLM 이 설계한 지식 구조의 편향

더 미묘한 문제는 지식의 '뼈대' 자체를 LLM이 만든다는 점이다. 페이지 제목을 어떻게 붙일지, 어떤 개념을 어떤 개념 아래에 둘지, 무엇을 상호 참조할지 — 이 모든 결정은 모델의 학습 데이터에 내재된 세계관을 반영한다. 영어권 자료로 주로 훈련된 모델이 만든 한국사 위키의 구조는, 한국인 편집자가 만들었다면 결코 나오지 않았을 방식으로 구획될 수 있다. 지식의 '내용'만이 아니라 지식의 '지도'까지 외주화하는 것 — 그 함의는 생각보다 무겁다.

IV. 반론 — '온디맨드 RAG'면 충분하지 않은가?

비평가들의 목소리 중 가장 설득력 있는 것은 '과잉 설계(over-engineering)'라는 지적이다.

그들은 묻는다. 정말로 모든 것을 미리 컴파일해둬야 하는가? 질문이 들어올 때마다 충분히 잘 설계된 RAG 파이프라인이 원본을 검색해 답하는 것으로 충분하지 않은가? 최신 LLM의 컨텍스트 창이 100만 토큰을 돌파한 지금, '지식을 미리 정리해둔다'는 전통적 필요 자체가 약화된 것 아닌가?

이 반론은 진지하게 받아들여야 한다. 카파시 본인도 LLM Wiki가 모든 경우의 해답이라고 주장한 적이 없다. 그가 드러낸 것은, 더 정확하게 말하자면, '지식 관리'와 '정보 검색'이 본래 다른 문제였다는 사실이다. RAG는 후자에 최적화되어 있다. 하지만 연구자·저술가·법률가의료진처럼 동일한 지식 체계를 수년간 누적해가며 작업하는 사람들에게는, 전자 — 축적되는 지식 — 가 여전히 본질적 요구다.

V. 보완 방향 — '반자동 큐레이션'이라는 중간지대

LLM Wiki의 잠재력을 살리면서 위험을 통제하기 위한 실용적 방향은 이미 여러 갈래로 나타나고 있다.

① 인간 개입 루프(Human-in-the-Loop)

에이전트가 위키에 중요한 변경을 가하기 전 인간의 승인을 거치게 하는 것이다. 이는 오류 증폭을 막는 가장 확실한 제동장치다. 완전 자동화를 포기하는 대신 신뢰성을 사는 것. 대부분의 실제 도입 현장에서 결국 선택되는 타협점이다.

② 하이브리드 아키텍처

위키와 RAG를 배타적 대안이 아니라 상보적 층위로 본다. 안정된 핵심 지식은 위키에 컴파

일해두고, 최신성이 중요한 영역이나 검증이 필요한 사실은 RAG로 원본을 재참조한다. 이 이중 구조는 '느린 뇌'와 '빠른 뇌'를 분리한 인간의 인지 구조와도 닮아 있다.

③ 망각 메커니즘(Forgetting by Design)

모든 지식을 영구 보존하려는 욕심을 버리는 것이다. 인간 기억은 강화되고, 약화되고, 통합되고, 잊혀진다. 이 사이클이 없으면 오래된 정보의 관성이 새 통찰을 가로막는다. 접근 빈도가 낮은 페이지를 자연스럽게 감쇠시키거나 아카이브로 밀어내는 설계는, 살아있는 지식 시스템에 필수적이다.

④ 출처 추적과 버전 관리

Git 기반 버전 관리와 메타데이터 주석은 선택이 아니라 필수다. 위키의 모든 주장이 어느 원본 문서의 어떤 구절에서 나왔는지 역추적 가능해야 하며, 변경 이력은 감사 가능해야 한다. 이것 없이 LLM Wiki는 감시받지 않는 블랙박스로 전락할 수 있다.

VI. 맷으며, 검색되는 지식에서 축적되는 지식으로

카파시의 LLM Wiki 제안은 완성된 아키텍처가 아니다. 그것은 하나의 질문이다. 우리는 지난 몇 년간 '더 많이 검색하고 더 많이 생성하는' 방향으로 AI를 밀어붙여 왔다. 그 결과 우리는 놀라운 성능을 얻었지만, 지식은 점점 더 파편화되고 일회적이 되었다. 매 대화마다 처음부터 다시 시작해야 하는 시스템을, 우리는 정말 '지능'이라고 부를 수 있는가?

LLM Wiki는 이 질문에 대한 한 가지 응답이다. 그것은 AI에게 '기억하는 방법'을 돌려주려는 시도이며, 동시에 인간에게 '지식의 주인 자리'를 되돌려주려는 시도이기도 하다. 물론 현재

의 형태 그대로 대규모 상용 시스템에 바로 이식되기는 어려울 것이다. 오류 누적, 비용, 편향이라는 세 개의 암초는 실재하는 위험이다.

그러나 향후 1~2년 사이 이 아이디어는 '완전 자동화된 위키'가 아닌, '반자동 지식 큐레이션 도구'의 형태로 재구성되어 확산될 가능성이 높다. 개인용 제2의 뇌(Second Brain), 기업용 내부 지식 허브, 연구 그룹의 공유 지식 베이스 — 소규모·고가치·검증 가능한 영역부터 LLM Wiki적 접근이 스며들 것이다.

분명한 것은 이것이다. **RAG가 답이 아니었다는 것이 아니라, RAG만으로는 충분하지 않다는 것.** 그리고 지식이란 원래, 검색되는 것이 아니라 축적되고 재조직되고 때로는 잊혀지는 살아 있는 과정이었다는 사실 — 카파시가 우리에게 다시 상기시킨 것은 바로 그것이다.

Dennis Kim (김호광)

gameworker@gmail.com · github.com/gameworkerkim